NÓDULO: Arquitetura e Soberania em Inteligência Artificial Acadêmica

1. Missão e Propósito

O **NÓDULO** é uma plataforma educacional de Inteligência Artificial de código aberto, concebida com um princípio inegociável: **a soberania tecnológica e a privacidade dos dados institucionais**.

Em um cenário onde universidades e empresas enviam dados sensíveis (pesquisas inéditas, materiais didáticos protegidos por direitos autorais e informações de alunos) para APIs proprietárias fechadas (como OpenAI ou Google), o NÓDULO propõe uma arquitetura modular onde a instituição detém o controle absoluto sobre a "memória" do sistema, garantindo que nenhum dado acadêmico seja utilizado para treinar modelos comerciais de terceiros.

2. Esclarecimento Fundamental: Como a IA Aprende vs. Como a IA Lê

Para compreender o NÓDULO, é crucial desfazer um equívoco comum sobre Inteligência Artificial: a diferença entre Treinamento (Fine-Tuning) e Geração Aumentada por Recuperação (RAG - Retrieval-Augmented Generation).

- **Treinamento (O que o NÓDULO *NÃO* faz com os seus dados):** Treinar um modelo de IA significa alterar os seus pesos matemáticos internos. É um processo que exige supercomputadores rodando por meses e custa milhões de dólares. Se treinássemos o modelo com os PDFs da universidade, esses PDFs ficariam "fundidos" na mente da IA, sendo impossível apagá-los depois ou controlar exatamente o que ela lembra.
- **O método RAG (O que o NÓDULO faz):** Imagine que o modelo de IA (Qwen 2.5 Coder) é um aluno brilhante, com raciocínio perfeito, mas que está a fazer uma prova de "livro aberto". O aluno não decorou o livro da universidade (não foi treinado nele). Em vez disso, quando o usuário faz uma pergunta, o NÓDULO atua como um bibliotecário: vai ao nosso cofre seguro (o banco de dados),

pega apenas as páginas relevantes do livro e entrega ao aluno (IA) dizendo: *"Responda à pergunta baseando-se estritamente neste texto"*.

Conclusão de Segurança: O modelo de IA nunca é treinado com os dados sensíveis. Ele apenas os "lê" temporariamente na memória RAM para formular a resposta, e depois esquece. O conhecimento real vive exclusivamente no Banco de Dados da universidade.

3. A Arquitetura Modular (A Tríade do Sistema)

O NÓDULO foi desenhado para separar completamente a Interface, a Lógica/Memória e o Motor de Processamento. Isso garante que a infraestrutura não fique refém de um único fornecedor de nuvem.

A. Frontend (A Interface do Usuário)

- **Tecnologia:** Next.js (React) e Tailwind CSS.
- **Função:** Ponto de interação dos estudantes e administradores. Mantém um design focado e acadêmico. Ele não processa dados pesados, apenas exibe a interface e comunica com o Backend via APIs REST e conexões em tempo real (Server-Sent Events).

B. Backend e Cofre de Dados (O Cérebro Central)

- **Tecnologias:** Go (Golang) e PostgreSQL.
- **Função:** É aqui que a soberania acontece. O servidor em Go gerencia a autenticação, permissões, histórico de mensagens e tarefas em segundo plano.
- **O Banco de Dados (PostgreSQL + pgvector):** Onde os dados sensíveis vivem. Quando um professor faz o upload de um slide ou PDF, o arquivo nunca vai para a nuvem pública. Ele fica guardado neste banco de dados, que pode estar instalado num servidor físico dentro da própria universidade.

C. Motor de Inferência (O Processador Matemático)

- **Tecnologia:** Ollama (rodando Qwen 2.5 Coder 32B e Nomic-Embed-Text).
- **Função:** O processador matemático descartável. Como a IA exige muito processamento gráfico (GPU), o NÓDULO permite que este motor seja executado em qualquer lugar. Pode rodar no notebook do aluno, num servidor local da universidade, ou numa placa A100 no Google Colab/RunPod. Se o Colab for desligado, **zero dados institucionais são perdidos**, pois a IA é apenas

uma "calculadora de palavras". A memória da calculadora está no PostgreSQL (Item B).

4. O Pipeline de Ingestão de Conhecimento (Como os PDFs viram inteligência)

Quando um administrador sobe um arquivo longo (ex: um PDF de 500 páginas), o sistema não pode enviar tudo de uma vez para a IA. O NÓDULO utiliza um fluxo assíncrono (Job Worker) em Go:

1. **Extração:** O Go lê o PDF e remove imagens e formatações, extraindo apenas o texto puro.
2. **Fragmentação (Chunking):** O texto de 500 páginas é cortado em milhares de pequenos fragmentos (cerca de um ou dois parágrafos cada).
3. **Vetorização (Embeddings):** Aqui entra o modelo secundário (`nomic-embed-text`). Ele lê cada parágrafo e o converte numa coordenada matemática gigante (um vetor de 768 dimensões). Textos com significados semelhantes ficam matematicamente próximos.
4. **Armazenamento:** Esses vetores e os textos originais são guardados no PostgreSQL. O arquivo está oficialmente indexado.

5. O Ciclo de Vida de uma Pergunta (O Fluxo de Chat)

O que acontece em milissegundos quando um estudante pergunta: *"Quais as regras da árvore Rubro-Negra segundo a apostila?"*:

1. **Vetorização da Pergunta:** O Go pega a pergunta e converte num vetor matemático usando o mesmo modelo (`nomic-embed-text`).
2. **Busca Semântica (pgvector):** O PostgreSQL realiza um cálculo de distância geométrica e encontra os 3 ou 4 fragmentos da apostila que estão mais "próximos" (semanticamente similares) à pergunta do estudante.
3. **Construção do Prompt de Contexto:** O Go junta a pergunta do estudante com os 4 fragmentos recuperados do banco e cria uma instrução invisível para a IA: *"Você é o assistente NÓDULO. Responda à pergunta abaixo usando apenas este contexto da nossa apostila. Contexto: [Texto dos 4 fragmentos]"*.
4. **Inferência RAG:** O pacote é enviado ao modelo principal (Qwen 32B na GPU A100). A IA lê o contexto, compreende a resposta, e começa a digitar de volta.

5. **Streaming:** O Go recebe a resposta palavra por palavra (tokens) e transmite em tempo real para a tela do estudante (Next.js).

6. Escalabilidade Assíncrona e Goroutines

Para lidar com arquivos `.zip` pesados ou múltiplos alunos acessando ao mesmo tempo, o Backend em Go utiliza **Goroutines** e **Channels**. Em vez de travar o servidor esperando um PDF ser lido, o NÓDULO coloca o trabalho numa fila invisível (Fila de Tarefas). Trabalhadores silenciosos (Workers) vão processando as páginas em segundo plano e atualizando o painel de administração em tempo real. Isso confere ao sistema características de performance de nível corporativo, mantendo baixo consumo de memória RAM no servidor central.

7. Conclusão da Arquitetura

O NÓDULO prova que é possível ter IA de ponta na educação sem vender os dados institucionais para corporações. Separar os dados (PostgreSQL local) do raciocínio matemático (GPU conectada dinamicamente) é o que permite à universidade manter seu patrimônio intelectual seguro, enquanto usufrui das mais modernas ferramentas de Inteligência Artificial do mundo.

8. Escalonamento e Prova de Conceito: O Nó de Inferência Descartável

Para demonstrar a escalabilidade do **NÓDULO** sem comprometer a soberania institucional, a arquitetura permite a delegação do processamento pesado para aceleradores de hardware externos (como GPUs A100 no Google Colab ou plataformas sob demanda).

Através de um script de provisionamento automatizado (presente no acervo do projeto sob o nome `infra_nuvem.md`), a plataforma instancia o motor de inferência temporário. O script cuida da instalação do ambiente, descarregamento dos modelos de alta capacidade (32B parâmetros) e estabelece um túnel criptografado reverso (Cloudflare Tunnel) comunicando-se de forma segura com o backend residente na instituição.

Esta abordagem consolida a premissa fundamental do NÓDULO: **a nuvem atua estritamente como uma calculadora matemática descartável**. Todo o envio de

contexto (RAG) ocorre em tempo real e permanece retido unicamente na memória volátil (VRAM) da GPU externa. Ao encerrar a sessão na nuvem, o hardware virtual é destruído, não restando qualquer rastro ou resíduo de conhecimento acadêmico. Isso comprova de modo definitivo a eficácia da separação entre a memória persistente (que vive no PostgreSQL local do NÓDULO) e o processador de inferência, garantindo blindagem institucional aos dados.